

FEATURE ENGINEERING

Complete Assignment – Questions & Answers

Q1. What is a Parameter?

A **parameter** is a value that describes some characteristic of a population or controls a model.

In Machine Learning, parameters are values learned by the model from training data.

Example

In Linear Regression:

$$y=mx+c$$

Here:

- m = slope/weight
- c = intercept/bias

The model learns these values during training.

Python Example

```
from sklearn.linear_model import LinearRegression

import numpy as np
```

```

X = np.array([[1], [2], [3], [4]])

y = np.array([2, 4, 6, 8])

model = LinearRegression()

model.fit(X, y)

print("Coefficient:", model.coef_)

print("Intercept:", model.intercept_)

```

Easy Definition

Parameter = A value learned or used by a model to describe a relationship in data.

Q2. What is Correlation?

Correlation measures the strength and direction of the relationship between two variables.

The correlation coefficient usually ranges from:

$$-1 \leq r \leq 1$$

**Correlati
on**

Meaning

+1	Perfect positive correlation
0	No linear correlation
-1	Perfect negative correlation

Example

If study hours increase and exam marks also increase, they may have a positive correlation.

Python Example

```
import pandas as pd

data = {

    "Study_Hours": [1, 2, 3, 4, 5],

    "Marks": [40, 50, 60, 70, 80]

}

df = pd.DataFrame(data)

print(df.corr())
```

Easy Definition

Correlation = Measure of how strongly two variables are related.

Q3. What Does Negative Correlation Mean?

Negative correlation means that when one variable increases, the other variable tends to decrease.

The correlation coefficient is between:

-1 and 0

Example

Suppose:

- Price increases
- Demand decreases

Then price and demand may have negative correlation.

Python Example

```
import pandas as pd
```

```
data = {  
  
    "Price": [10, 20, 30, 40, 50],  
  
    "Demand": [100, 80, 60, 40, 20]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print(df.corr())
```

The correlation will be close to -1 .

Easy Definition

Negative correlation = One variable increases while the other tends to decrease.

Q4. Define Machine Learning. What Are the Main Components in Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence that allows computers to learn patterns from data and make predictions or decisions without being explicitly programmed for every situation.

Main Components

1. **Data** – Information used by the model.
2. **Features** – Input variables.
3. **Target** – Output variable to predict.
4. **Model** – Algorithm that learns patterns.
5. **Training** – Process of learning from data.
6. **Loss/Cost Function** – Measures prediction error.

7. **Optimizer** – Helps improve model parameters.
8. **Evaluation** – Measures model performance.
9. **Prediction** – Model generates output for new data.

Basic ML Workflow

Data



Data Cleaning



EDA



Feature Engineering



Train-Test Split



Model Training



Model Evaluation



Prediction

Q5. How Does Loss Value Help in Determining Whether the Model Is Good or Not?

A **loss function** measures the difference between the actual value and the predicted value.

Generally:

$$\text{Loss} = \text{Difference}(\text{Actual}, \text{Predicted})$$

A lower loss usually indicates that the model's predictions are closer to the actual values.

Example

Actual value:

100

Predicted value:

95

The error is smaller than if the model predicted:

60

Common Loss Functions

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Binary Cross-Entropy
- Categorical Cross-Entropy

Python Example

```
from sklearn.metrics import mean_squared_error

actual = [100, 200, 300]

predicted = [90, 210, 290]

loss = mean_squared_error(actual, predicted)

print("MSE Loss:", loss)
```

Important Point

A lower training loss alone does **not** guarantee a good model. We should also check validation/test performance to detect overfitting.

Q6. What Are Continuous and Categorical Variables?

Continuous Variables

Continuous variables can take numerical values over a range.

Examples

- Height
- Weight
- Temperature
- Salary
- Age
- Distance

Example:

Height = 172.5 cm

Weight = 68.7 kg

Categorical Variables

Categorical variables represent groups or categories.

Examples

- Gender
- City
- Department
- Product Type
- Color

Example:

City = Jaipur

Color = Red

Department = Sales

Difference

Continuous	Categorical
Numerical measurement	Categories/groups
Can have decimal values	Usually labels
Height, weight	City, gender
Often scaled	Often encoded

Q7. How Do We Handle Categorical Variables in Machine Learning? What Are the Common Techniques?

Machine Learning models generally require numerical input. Therefore, categorical variables need to be converted into numerical form.

Common Techniques

1. Label Encoding

Each category is assigned a number.

```
from sklearn.preprocessing import LabelEncoder
```

```
data = ["Red", "Blue", "Green", "Red"]
```

```
encoder = LabelEncoder()
```

```
encoded = encoder.fit_transform(data)
```

```
print(encoded)
```

Example:

Blue → 0

Green → 1

Red → 2

2. One-Hot Encoding

Creates separate binary columns for each category.

```
import pandas as pd
```

```
df = pd.DataFrame({
```

```
    "Color": ["Red", "Blue", "Green", "Red"]
```

```
})
```

```
encoded_df = pd.get_dummies(df, columns=["Color"])
```

```
print(encoded_df)
```

3. Ordinal Encoding

Used when categories have a meaningful order.

Example:

Low → 1

Medium → 2

High → 3

```
from sklearn.preprocessing import OrdinalEncoder
```

```
data = [["Low"], ["Medium"], ["High"], ["Low"]]
```

```
encoder = OrdinalEncoder()
```

```
print(encoder.fit_transform(data))
```

Common Techniques

- Label Encoding
 - One-Hot Encoding
 - Ordinal Encoding
 - Target Encoding
 - Frequency Encoding
-

Q8. What Do You Mean by Training and Testing a Dataset?

Training means using data to teach the Machine Learning model.

Testing means evaluating the trained model on unseen data.

Example

Suppose we have 1000 records:

800 records → Training

200 records → Testing

The model learns from the 800 training records and is evaluated on the 200 unseen testing records.

Why?

It helps determine whether the model can **generalize to new data**.

Q9. What is sklearn.preprocessing?

`sklearn.preprocessing` is a module in Scikit-learn that provides tools for **transforming and preprocessing data** before using it in Machine Learning models.

Common Classes

- `StandardScaler`
- `MinMaxScaler`
- `RobustScaler`
- `Normalizer`
- `LabelEncoder`
- `OneHotEncoder`
- `OrdinalEncoder`
- `PolynomialFeatures`

Example

```
from sklearn.preprocessing import StandardScaler
```

```
data = [[10], [20], [30], [40], [50]]
```

```
scaler = StandardScaler()
```

```
scaled_data = scaler.fit_transform(data)
```

```
print(scaled_data)
```

Q10. What is a Test Set?

A **test set** is a portion of the dataset that is kept separate from training data and used to evaluate the final performance of a trained Machine Learning model.

Example

```
Dataset = 1000 records
```

```
Training Set = 800
```

```
Test Set      = 200
```

The model does not learn from the test set.

Purpose

The test set helps determine how well the model performs on **unseen data**.

Q11. How Do We Split Data for Model Fitting (Training and Testing) in Python?

We can use `train_test_split()` from Scikit-learn.

Python / Google Colab Code

```
from sklearn.model_selection import train_test_split
```

```
import pandas as pd
```

```
df = pd.DataFrame({  
  
    "Age": [20, 22, 25, 28, 30, 32, 35, 38, 40, 42],  
  
    "Salary": [20000, 22000, 25000, 28000, 30000,  
               32000, 35000, 38000, 40000, 42000]  
})
```

```
X = df[["Age"]]
```

```
y = df["Salary"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
  
    X,  
  
    y,  
  
    test_size=0.2,
```

```
        random_state=42

    )

    print("Training Data:")

    print(X_train)

    print("\nTesting Data:")

    print(X_test)
```

Explanation

```
test_size=0.2
```

means 20% data is used for testing and 80% for training.

```
random_state=42
```

ensures reproducible splitting.

Q12. How Do You Approach a Machine Learning Problem?

A systematic approach is important when solving an ML problem.

Steps

1. Understand the Problem

Identify:

- Business problem
- Objective
- Target variable
- Success criteria

2. Collect Data

Obtain relevant data.

3. Clean the Data

Handle:

- Missing values
- Duplicates
- Incorrect data
- Outliers

4. Perform EDA

Analyze:

- Distributions
- Relationships
- Correlation
- Outliers

5. Feature Engineering

Create, transform, select, or encode useful features.

6. Split Data

Separate training and testing data.

7. Select Model

Choose an appropriate algorithm.

Examples:

- Linear Regression
- Logistic Regression
- Decision Tree
- Random Forest
- SVM

8. Train Model

Use training data.

9. Evaluate Model

Use suitable metrics.

10. Tune Model

Optimize hyperparameters.

11. Test and Deploy

Evaluate on unseen data and deploy if appropriate.

Q13. Why Do We Have to Perform EDA Before Fitting a Model to the Data?

EDA (Exploratory Data Analysis) helps us understand the dataset before building a Machine Learning model.

EDA Helps Identify

- Missing values
- Duplicate records
- Outliers
- Incorrect values
- Data distributions
- Relationships between variables
- Correlation
- Categorical variables
- Potential data leakage

Example

```
import pandas as pd

df = pd.DataFrame({

    "Age": [20, 22, 25, 30, 35],

    "Salary": [20000, 25000, 30000, 40000, 50000]

})

print(df.head())

print(df.info())

print(df.describe())
```

```
print(df.isnull().sum())
```

```
print(df.corr())
```

Importance

Without EDA, we may train a model on poor-quality or incorrectly processed data.

Easy Definition

EDA = Understanding and exploring the data before building the model.

Q14. What is Correlation?

Correlation is a statistical measure that describes the **strength and direction of a relationship between two variables**.

$$-1 \leq r \leq 1$$

Example

```
import pandas as pd
```

```
df = pd.DataFrame({  
    "Hours": [1, 2, 3, 4, 5],  
    "Marks": [40, 50, 60, 70, 80]  
})
```

```
print(df["Hours"].corr(df["Marks"]))
```

The result will be close to +1, showing strong positive correlation.

Q15. What Does Negative Correlation Mean?

Negative correlation means that two variables move in opposite directions.

For example:

Price ↑ → Demand ↓

The correlation coefficient is negative.

```
import pandas as pd
```

```
df = pd.DataFrame({  
    "Price": [10, 20, 30, 40, 50],  
    "Demand": [100, 80, 60, 40, 20]  
})
```

```
print(df["Price"].corr(df["Demand"]))
```

The result will be close to -1 .

Q16. How Can You Find Correlation Between Variables in Python?

There are several ways to calculate correlation.

Using Pandas

```
import pandas as pd

df = pd.DataFrame({

    "Age": [20, 25, 30, 35, 40],

    "Salary": [20000, 30000, 40000, 50000, 60000],

    "Experience": [1, 2, 4, 6, 8]

})

print(df.corr())
```

Correlation Between Two Specific Columns

```
print(df["Age"].corr(df["Salary"]))
```

Using NumPy

```
import numpy as np
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
correlation = np.corrcoef(x, y)
```

```
print(correlation)
```

Correlation Heatmap

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
```

```
plt.title("Correlation Heatmap")
```

```
plt.show()
```

Q17. What is Causation? Explain Difference Between Correlation and Causation with an Example.

Causation means that a change in one variable directly causes a change in another variable.

Correlation vs Causation

Correlation	Causation
Variables are associated	One variable causes another
Does not prove cause	Shows cause-effect relationship
May be due to another variable	Direct causal relationship

Example

Suppose:

Ice cream sales ↑

Drowning cases ↑

These two variables may be correlated.

But buying ice cream does **not** cause drowning.

A third variable—**hot weather**—can increase both:

Hot Weather

↓ ↓

Ice Cream Swimming

Sales ↑

↓

Drowning Risk

Therefore:

Correlation does not necessarily mean causation.

Q18. What is an Optimizer? What Are Different Types of Optimizers? Explain Each with an Example.

An **optimizer** is an algorithm that adjusts a model's parameters to minimize the loss function.

In simple terms:

Prediction

↓

Calculate Loss

↓

Optimizer adjusts parameters

↓

Lower Loss

1. Gradient Descent

Gradient Descent updates parameters in the direction that reduces the loss.

Formula:

$$\theta = \theta - \alpha \nabla J(\theta)$$

Where:

- θ = model parameters
- α = learning rate
- ∇J = gradient of loss

2. Stochastic Gradient Descent (SGD)

SGD updates parameters using individual training samples or small batches.

```
from tensorflow.keras.optimizers import SGD
```

```
optimizer = SGD(learning_rate=0.01)
```

3. Adam

Adam combines ideas from momentum and adaptive learning rates.

```
from tensorflow.keras.optimizers import Adam
```

```
optimizer = Adam(learning_rate=0.001)
```

Adam is widely used in Deep Learning.

4. RMSprop

RMSprop adapts the learning rate using a moving average of squared gradients.

```
from tensorflow.keras.optimizers import RMSprop
```

```
optimizer = RMSprop(learning_rate=0.001)
```

Comparison

Optimizer	Main Idea
Gradient Descent	Move toward minimum loss

SGD	Update using samples/mini-batches
Adam	Adaptive learning + momentum
RMSprop	Adaptive learning rate

Easy Definition

Optimizer = Algorithm that updates model parameters to reduce loss.

Q19. What is sklearn.linear_model?

`sklearn.linear_model` is a Scikit-learn module containing linear models for regression and classification.

Important Models

- `LinearRegression`
- `LogisticRegression`
- `Ridge`
- `Lasso`
- `ElasticNet`
- `SGDRegressor`

Example

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
X = [[1], [2], [3], [4]]
```

```
y = [2, 4, 6, 8]
```

```
model.fit(X, y)
```

```
print(model.predict([[5]]))
```

Output will be approximately:

```
[10.]
```

Q20. What Does model.fit() Do? What Arguments Must Be Given?

`model.fit()` trains a Machine Learning model using training data.

Syntax

```
model.fit(X_train, y_train)
```

Arguments

`X_train`

Input features.

`y_train`

Target/output values.

Example

```
from sklearn.linear_model import LinearRegression
```

```
X_train = [[1], [2], [3], [4]]
```

```
y_train = [2, 4, 6, 8]
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

During `fit()`, the model learns parameters from the training data.

Easy Definition

`fit()` = Train the model using input features and target values.

Q21. What Does model.predict() Do? What Arguments Must Be Given?

`model.predict()` uses a trained model to make predictions for new input data.

Syntax

```
model.predict(X_test)
```

Example

```
from sklearn.linear_model import LinearRegression
```

```
X_train = [[1], [2], [3], [4]]
```

```
y_train = [2, 4, 6, 8]
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
prediction = model.predict([[5]])
```

```
print("Prediction:", prediction)
```

Output:

```
Prediction: [10.]
```

Important

`predict()` requires the input features in the same structure used during training.

Easy Definition

`predict()` = Use the trained model to predict output for new data.

Q22. What Are Continuous and Categorical Variables?

Continuous Variables

Variables that represent measurable numerical quantities.

Examples:

```
Height = 175.5
```

```
Weight = 68.2
```

```
Temperature = 32.7
```

Categorical Variables

Variables that represent categories or groups.

Examples:

City = Jaipur

Color = Blue

Department = IT

Summary

Continuous → Numerical measurements

Categorical → Categories/labels

Q23. What is Feature Scaling? How Does It Help in Machine Learning?

Feature scaling is the process of transforming numerical features so that they have comparable scales.

Example

Suppose:

Age = 20-60

Salary = 20,000-100,000

Salary has a much larger numerical scale.

Some algorithms can be affected by these differences.

Common Scaling Methods

1. Standardization
2. Min-Max Scaling
3. Robust Scaling

Standardization

Formula:

$$Z = \frac{X - \mu}{\sigma}$$

After standardization:

- Mean ≈ 0
- Standard deviation ≈ 1

Python

```
from sklearn.preprocessing import StandardScaler
```

```
data = [[10, 1000],
```

```
[20, 2000],  
  
[30, 3000],  
  
[40, 4000]]  
  
scaler = StandardScaler()  
  
scaled_data = scaler.fit_transform(data)  
  
print(scaled_data)
```

Why Scaling Is Useful?

It is especially important for algorithms such as:

- K-Nearest Neighbors
- K-Means
- Support Vector Machines
- Logistic Regression
- Linear Regression in some settings
- Neural Networks
- PCA

Tree-based models generally do not require feature scaling.

Q24. How Do We Perform Scaling in Python?

We can use Scikit-learn's preprocessing classes.

StandardScaler

```
from sklearn.preprocessing import StandardScaler
```

```
X = [[10, 100],  
      [20, 200],  
      [30, 300],  
      [40, 400]]
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
print(X_scaled)
```

MinMaxScaler

Min-Max Scaling generally transforms values to a specified range, commonly 0 to 1.

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
print(X_scaled)
```

RobustScaler

Useful when the data contains significant outliers.

```
from sklearn.preprocessing import RobustScaler
```

```
scaler = RobustScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
print(X_scaled)
```

Important Rule

Fit the scaler on training data only:

```
scaler.fit(X_train)
```

```
X_train_scaled = scaler.transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

This prevents information from the test set from leaking into the training process.

Q25. What is sklearn.preprocessing?

`sklearn.preprocessing` provides preprocessing and transformation tools for Machine Learning datasets.

Important Tools

```
from sklearn.preprocessing import (  
  
    StandardScaler,  
  
    MinMaxScaler,  
  
    RobustScaler,  
  
    LabelEncoder,  
  
    OneHotEncoder,  
  
    OrdinalEncoder  
  
)
```

Example

```
from sklearn.preprocessing import StandardScaler
```

```
X = [[10], [20], [30], [40], [50]]
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
print(X_scaled)
```

Main Purpose

It helps with:

- Feature scaling
 - Encoding
 - Normalization
 - Data transformation
 - Feature generation
-

Q26. How Do We Split Data for Model Fitting (Training and Testing) in Python?

The most common method is `train_test_split()`.

```
from sklearn.model_selection import train_test_split
```

```
X = [  
  
    [1], [2], [3], [4], [5],  
  
    [6], [7], [8], [9], [10]  
  
]
```

```
y = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
  
    X,  
  
    y,  
  
    test_size=0.2,  
  
    random_state=42  
  
)
```

```
print("X_train:", X_train)
```

```
print("X_test:", X_test)
```

```
print("y_train:", y_train)
```

```
print("y_test:", y_test)
```

Common Split

80% → Training

20% → Testing

Another common choice is:

70% → Training

30% → Testing

The appropriate split depends on dataset size and problem requirements.

Q27. Explain Data Encoding.

Data encoding is the process of converting categorical data into a numerical representation that Machine Learning algorithms can process.

For example:

Color

Red

Blue

Green

A Machine Learning model generally needs these categories represented numerically.

1. Label Encoding

```
from sklearn.preprocessing import LabelEncoder
```

```
colors = ["Red", "Blue", "Green", "Red"]
```

```
encoder = LabelEncoder()
```

```
encoded = encoder.fit_transform(colors)
```

```
print(encoded)
```

Possible representation:

Blue → 0

Green → 1

Red → 2

2. One-Hot Encoding

```
import pandas as pd

df = pd.DataFrame({
    "Color": ["Red", "Blue", "Green", "Red"]
})

encoded_df = pd.get_dummies(df, columns=["Color"])

print(encoded_df)
```

It creates columns such as:

Color_Blue

Color_Green

Color_Red

with 0/1 values.

3. Ordinal Encoding

Used when categories have a natural order.

Example:

Low

Medium

High

```
from sklearn.preprocessing import OrdinalEncoder
```

```
data = [  
    ["Low"],  
    ["Medium"],  
    ["High"],  
    ["Low"]  
]
```

```
encoder = OrdinalEncoder()

encoded = encoder.fit_transform(data)

print(encoded)
```

Encoding Comparison

Method	Suitable For
Label Encoding	Converting labels to numbers
One-Hot Encoding	Nominal categories
Ordinal Encoding	Ordered categories
Target Encoding	High-cardinality categorical features

Important Point

We should choose the encoding method according to the nature of the categorical variable. For nominal categories, assigning arbitrary numerical order can incorrectly imply that one category is greater than another.

GOOGLE COLAB – COMPLETE PRACTICAL EXAMPLE

The following code combines important Feature Engineering steps in one Google Colab example.

```
# Import libraries

import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.compose import ColumnTransformer

from sklearn.pipeline import Pipeline

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error

# Create sample dataset

df = pd.DataFrame({

    "Age": [22, 25, 28, 30, 35, 40, 42, 45, 50, 55],
```

```
"Experience": [1, 2, 4, 5, 8, 12, 14, 17, 20, 25],

"City": [

    "Jaipur", "Delhi", "Mumbai", "Jaipur", "Delhi",

    "Mumbai", "Jaipur", "Delhi", "Mumbai", "Jaipur"

],

"Salary": [

    25000, 30000, 35000, 40000, 50000,

    65000, 70000, 80000, 95000, 110000

]

})
```

```
# Display data
```

```
print("Dataset:")
```

```
display(df)
```

```
# Check missing values
```

```
print("Missing Values:")
```

```
print(df.isnull().sum())
```

```
# Correlation
```

```
print("\nCorrelation:")
```

```
print(df[["Age", "Experience", "Salary"]].corr())
```

```
# Define features and target
```

```
X = df.drop("Salary", axis=1)
```

```
y = df["Salary"]
```

```
# Identify columns
```

```
numerical_features = ["Age", "Experience"]
```

```
categorical_features = ["City"]
```

```
# Create preprocessing pipeline
```

```
preprocessor = ColumnTransformer(
```

```
    transformers=[
```

```
        ("num", StandardScaler(), numerical_features),
```

```
        ("cat", OneHotEncoder(handle_unknown="ignore"),  
categorical_features)
```

```
    ]
```

```
)
```

```
# Create complete ML pipeline
```

```
model = Pipeline(
```

```
    steps=[
```

```
        ("preprocessor", preprocessor),

        ("regressor", LinearRegression())

    ]

)


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,

    random_state=42

)


# Train model

model.fit(X_train, y_train)
```

```
# Make predictions

y_pred = model.predict(X_test)

# Evaluate model

mse = mean_squared_error(y_test, y_pred)

print("\nActual Values:")

print(y_test.values)

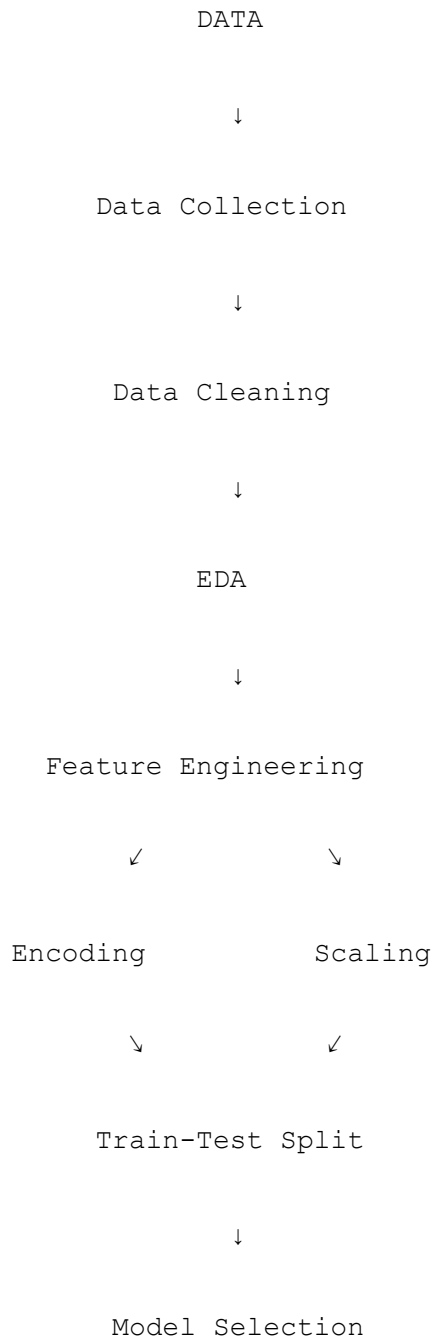
print("\nPredicted Values:")

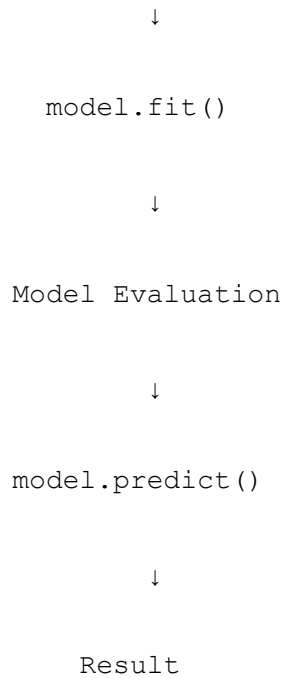
print(y_pred)

print("\nMean Squared Error:", mse)
```

COMPLETE MACHINE LEARNING WORKFLOW

For the questions in this assignment, remember this workflow:





IMPORTANT DEFINITIONS FOR EXAM

Parameter

A value that describes a population or is learned by a Machine Learning model.

Correlation

A measure of the strength and direction of the relationship between two variables.

Negative Correlation

When one variable increases, the other tends to decrease.

Machine Learning

A method that enables computers to learn patterns from data and make predictions or decisions.

Loss Function

A function that measures the error between actual and predicted values.

Continuous Variable

A numerical variable that can take values within a range.

Categorical Variable

A variable representing categories or groups.

Training Set

Data used to train the Machine Learning model.

Test Set

Unseen data used to evaluate the trained model.

Feature Scaling

Transforming numerical features to comparable scales.

Encoding

Converting categorical variables into numerical representations.

```
model.fit()
```

Trains the model using input features and target values.

```
model.predict()
```

Generates predictions using a trained model.

Optimizer

An algorithm that updates model parameters to minimize the loss function.

EDA

Exploratory Data Analysis is the process of understanding and analyzing data before modeling.

QUICK REVISION TABLE

Topic	Key Point
Parameter	Model/population value
Correlation	Relationship between variables
Negative Correlation	One increases, other decreases
ML	Learns patterns from data
Loss	Measures prediction error
Continuous	Numerical measurement
Categorical	Categories/labels
Training Set	Used to train model
Test Set	Used to evaluate model
<code>sklearn.preprocessing</code>	Data preprocessing tools

<code>train_test_split()</code>	Splits data
EDA	Understand data
Optimizer	Reduces loss
<code>sklearn.linear_model</code>	Linear ML models
<code>fit()</code>	Train model
<code>predict()</code>	Make predictions
Feature Scaling	Makes numerical scales comparable
Encoding	Converts categories to numbers

Final Key Point

Feature Engineering is the process of creating, transforming, selecting, scaling, and encoding features so that Machine Learning algorithms can learn useful patterns more effectively from the data.
